

Introduction to Digital Logic Design Lab

EECS 31L

Lab 1: Logic Block Design

Ali Mortada
19849621

January 26th, 2025

1 Objective

The purpose of this lab was to implement and test some basic logic blocks in Verilog including a Half Adder, Full Adders, and Multiplexers.

1.1 Half Adder

A Half Adder has two 1-bit inputs A and B and two 1-bit outputs Sum and Carry Out. It adds the values of the inputs and outputs the sum and carry out values to their respective terminals. Below is the block diagram in Fig. 1.

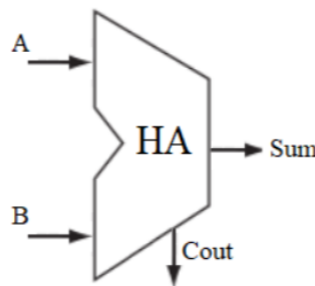


Figure 1: Block diagram for Half Adder.

1.2 1-bit Full Adder

A 1-bit Full Adder takes in three 1-bit inputs (A, B, and Carry In) and has two 1-bit outputs (Sum and Carry Out). It adds the values of the three inputs and outputs the sum and carry out values to their respective terminals. Below is the block diagram in Fig. 2.

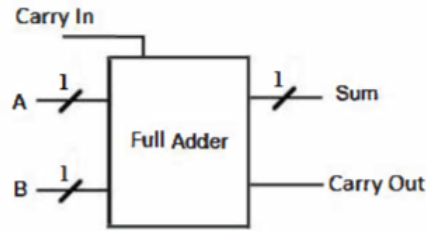


Figure 2: Block diagram for 1-bit Full Adder.

1.3 4-bit Full Adder

A 4-bit Full Adder takes in two 4-bit inputs A and B and a 1-bit input Carry In and gives a 4-bit output Sum and a 1-bit output Carry Out. It performs the same function as the 1-bit Full Adder, except that A, B, and Sum are now four bits wide. Below is the block diagram in Fig. 3.

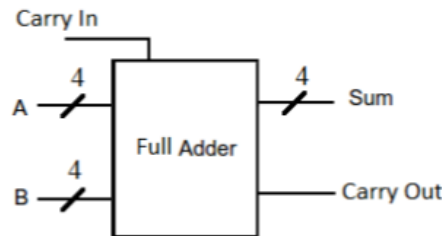


Figure 3: Block diagram for 4-bit Full Adder.

1.4 Multiplexer 2:1

A 2:1 Multiplexer takes in three 1-bit inputs D1, D2, and S, and gives one 1-bit output Y. The S input acts like a switch, and depending on its value, outputs either D1 or D2 to Y. In this case, when the value of S is equal to zero, the output is the value of D1, and when the value of S is equal to one, the output is the value of D2. Below is a block diagram in Fig. 4.

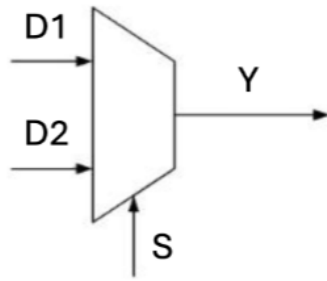


Figure 4: Block diagram for 2:1 Multiplexer.

1.5 Multiplexer 4:1

A 4:1 Multiplexer performs the same operation as a 2:1 Multiplexer but instead has four 1-bit inputs D1, D2, D3, and D4, and a 2-bit select input S. Since there are four inputs to switch between, S needs to be two bits wide to be able to cover each input. Below is a block diagram in Fig. 5.

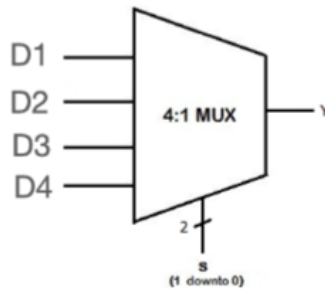


Figure 5: Block diagram for 4:1 Multiplexer.

2 Procedure

2.1 Half Adder

Below are the circuit diagram and truth table for the Half Adder in Figs. 6 and 7.

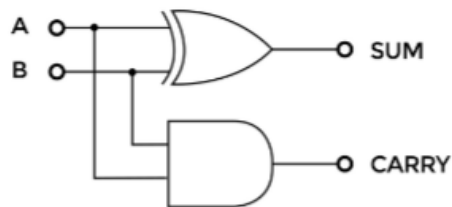


Figure 6: Circuit schematic for Half Adder.

Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 7: Truth table for Half Adder.

From the truth table in Fig. 7, the logic functions for the output are as follows:

$$\begin{aligned}\text{Sum} &= A \text{ XOR } B \\ \text{Carry} &= A \text{ AND } B\end{aligned}$$

This is evident in the circuit schematic shown in Fig. 6. Thus, the two statements used to implement the Half Adder in Verilog were:

```
assign Cout = A & B;
assign Sum = A ^ B;
```

2.2 1-bit Full Adder

Below are the circuit schematic and truth table for the 1-bit Full Adder in Figs. 8 and 9.

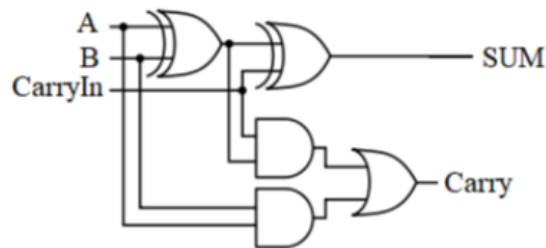


Figure 8: Circuit schematic for 1-bit Full Adder.

Full Adder – Truth Table				
Input			Output	
A	B	Carry in	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 9: Truth table for 1-bit Full Adder.

From the truth table in Fig. 9, the logic functions for the output are as follows:

$$\text{Sum} = A \text{ XOR } B \text{ XOR } C_{in}$$

$$C_{out} = (A \text{ AND } B) \text{ OR } (A \text{ AND } C_{in}) \text{ OR } (B \text{ AND } C_{in})$$

This logic can also be derived from the circuit schematic in Fig. 8. Much like the Half Adder, two statements were used to implement the functionality of the 1-bit Full Adder:

```
assign Sum = A ^ B ^ Cin;
assign Cout = (A & B) | (A & Cin) | (B & Cin);
```

2.3 4-bit Full Adder

Below is the circuit schematic for the 4-bit Full Adder in Fig. 10.

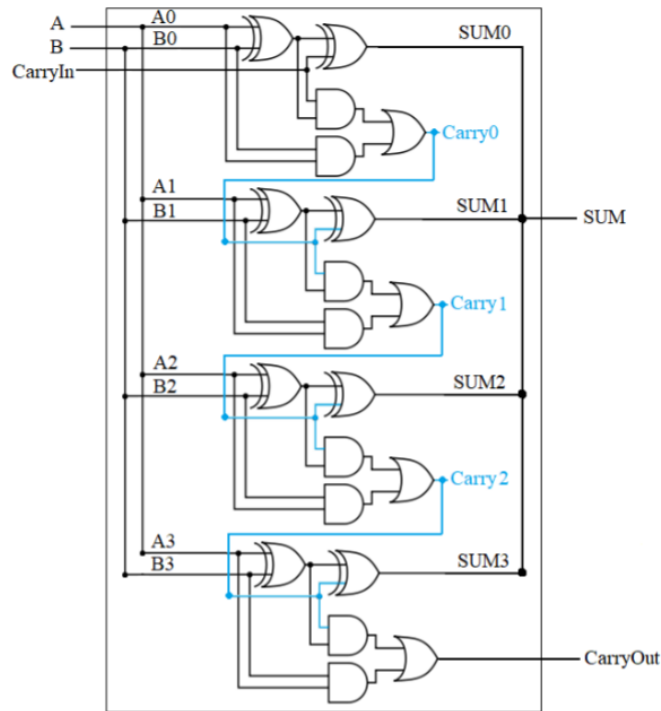


Figure 10: Circuit schematic for 4-bit Full Adder.

Although the functionality of the 4-bit Full Adder is similar to the 1-bit version, the implementation is slightly different due to the fact that the addition must be done bit-by-bit and each bit has a carry wire to the next most significant bit. There are two main ways to implement a 4-bit Full Adder: treating it as a standalone module and deriving the logic functions from the circuit schematic, or using the previously implemented 1-bit Full Adder and connecting four of them in series to form a 4-bit ripple-carry adder. For this lab, the first method was used as it was expected that each module would be written and tested in its own project. Thus, the logic functions and statements used to implement the logic were derived from the circuit schematic in Fig. 10 and are as follows: First, a 4-bit wire is defined for the internal Carry wires.

```
wire [3:0] Carry;
```

Next, the Sum values are computed for each bit as well as the Carry value for the next most significant bit.

```
// sum0, carry0
assign Sum[0] = (A[0] ^ B[0]) ^ Cin;
assign Carry[0] = (Cin & (A[0] ^ B[0])) | (A[0] & B[0]);

// sum1, carry1
assign Sum[1] = (A[1] ^ B[1]) ^ Carry[0];
assign Carry[1] = (Carry[0] & (A[1] ^ B[1])) | (A[1] & B[1]);

// sum2, carry2
assign Sum[2] = (A[2] ^ B[2]) ^ Carry[1];
```

```

assign Carry[2] = (Carry[1] & (A[2] ^ B[2])) | (A[2] & B[2]);

// sum3, carry3
assign Sum[3] = (A[3] ^ B[3]) ^ Carry[2];
assign Carry[3] = (Carry[2] & (A[3] ^ B[3])) | (A[3] & B[3]);

```

Finally, the most significant bit of the internal Carry wire is assigned to the external Carry Out bit.

```

assign Cout = Carry[3];

```

2.4 Multiplexer 2:1

Much like the 4-bit Full Adder, the 2:1 Multiplexer can be defined in multiple ways: using either combinatorial or sequential logic. To use sequential logic, the outputs of the modules must be defined as type **reg**; however, the module definition given in the lab manual has the output Y implicitly defined as type **wire**, meaning that sequential logic could not be used in this implementation. Thus, a single combinatorial statement using the ternary operator was used to define the logic of the multiplexer:

```

assign Y = (S ? D2 : D1);

```

If the select input S has the value '0', then the condition after the colon is evaluated and the input D1 is assigned to the output. If S has the value '1', then the condition after the question mark is evaluated and the input D2 is assigned to the output, thus adhering to the specification.

2.5 Multiplexer 4:1

The implementation of the 4:1 Multiplexer is similar to that of the 2:1 version, with the difference that there are four inputs to select from and that the select input S is two bits wide. Again, a single combinatorial statement was used to evaluate the logic of the multiplexer.

```

assign Y = (S == 2'b00) ? D1 :
           (S == 2'b01) ? D2 :
           (S == 2'b10) ? D3 :
           D4;

```

Here, the ternary operator is used again but is nested to allow the select input to choose between more than two inputs. If the first condition evaluates to **true**, then the first input is assigned to the output. Otherwise, the statement goes down the list of possibilities until all have been exhausted.

3 Simulation Results

3.1 Half Adder

For the Half Adder, four test cases exhaust the list of possible inputs and outputs (as seen in the truth table in Fig. 7). All these cases were used in the testbench, in the same order as the truth table. The simulation results are shown in Fig. 11 and are as expected.

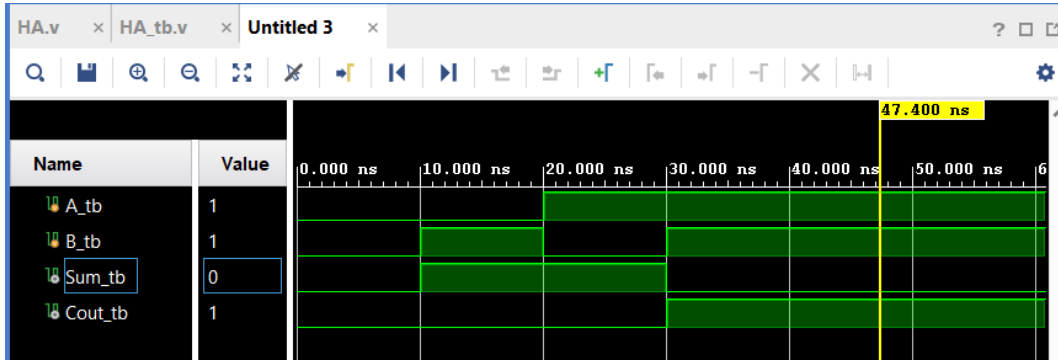


Figure 11: Half Adder simulation results.

3.2 1-Bit Full Adder

There are a list of eight possible inputs and outputs for the 1-bit Full Adder (as shown in Fig. 9). However, testing each input would be repetitive, so only three input cases were tested:

$$\begin{aligned}
 A = '0', B = '1', Cin = '0' \\
 A = '1', B = '1', Cin = '0' \\
 A = '1', B = '1', Cin = '1'
 \end{aligned}$$

The first test covers simple addition of one and zero, which should result in a sum of one and carry of zero. The second test covers the case where the carry out bit outputs a value of one, and the third test covers the case where both the sum and carry out bits output a value of one. The simulation results are shown in Fig. 12 and show that the implemented module correctly performs addition for 1-bit inputs.

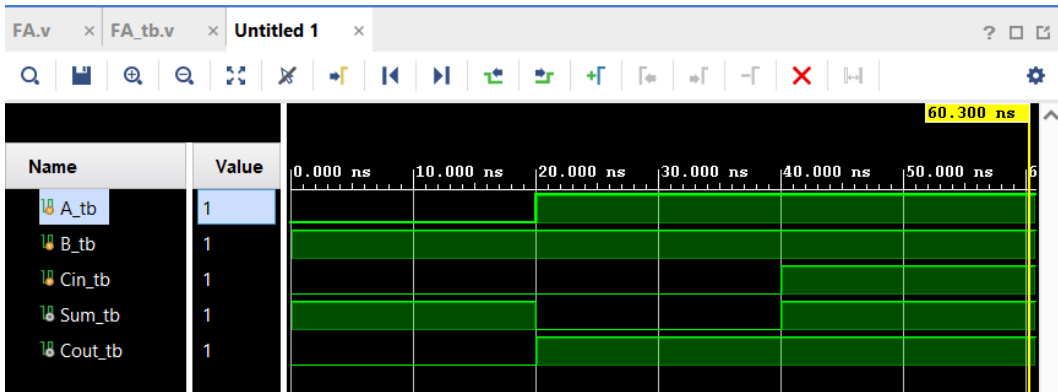


Figure 12: 1-bit Full Adder simulation results.

3.3 4-bit Full Adder

For the 4-bit Full Adder, there are a list of $2^4 = 16$ different input and output possibilities. Again, testing each one would be repetitive, so four input cases were tested:


```

A = '0110', B = '0100', Cin = '0'
A = '1000', B = '1001', Cin = '1'
A = '1110', B = '0010', Cin = '0'
A = '1010', B = '1011', Cin = '0'

```

All of these tests make sure that the module can do correct addition with and without a carry in bit. The simulation results are shown in Fig. 13 and show that the implemented module correctly performs addition for 4-bit inputs.

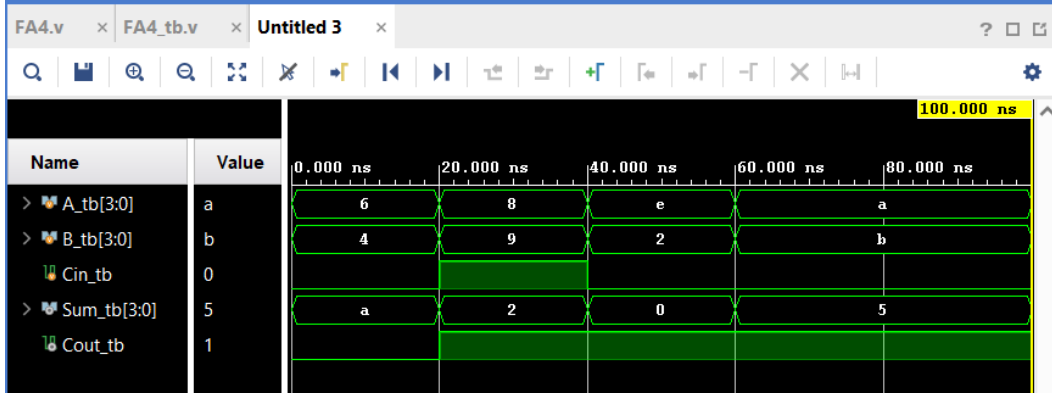


Figure 13: 4-bit Full Adder simulation results.

3.4 Multiplexer 2:1

For the 2:1 Multiplexer, two tests were run:

```

D1 = '0', D2 = '1', S = '0'
D1 = '0', D2 = '1', S = '1'

```

In the first test, the select bit is zero, so the the first input should be connected to the output, yielding a value of zero. In the second test, the select bit is one, so the second input should be connected to the output, yielding a value of one. Having D1 and D2 be distinct makes sure that it is clear which input is being connected to the output Y, thus confirming that the module works properly.

Fig. 14 shows the simulation results. The implemented module was able to correctly connect the output to the respective input based on the select signal.

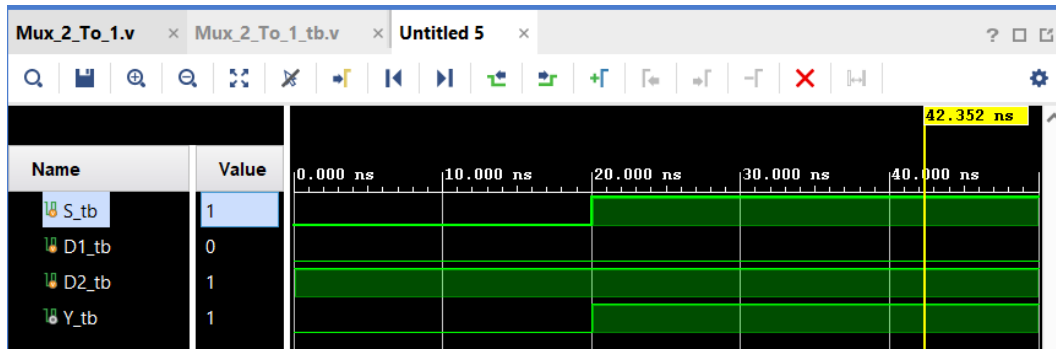


Figure 14: 2:1 Multiplexer simulation results.

3.5 Multiplexer 4:1

Four tests were used for the 4:1 Multiplexer:

$D1 = '0', D2 = '1', D3 = '0', D4 = '1', S = '00'$
 $D1 = '0', D2 = '1', D3 = '0', D4 = '1', S = '01'$
 $D1 = '0', D2 = '1', D3 = '0', D4 = '1', S = '10'$
 $D1 = '0', D2 = '1', D3 = '0', D4 = '1', S = '11'$

In order, the results should be zero, one, zero, one. Fig. 15 shows the simulation results. The implemented module was able to correctly connect the output to the respective input based on the select signal.

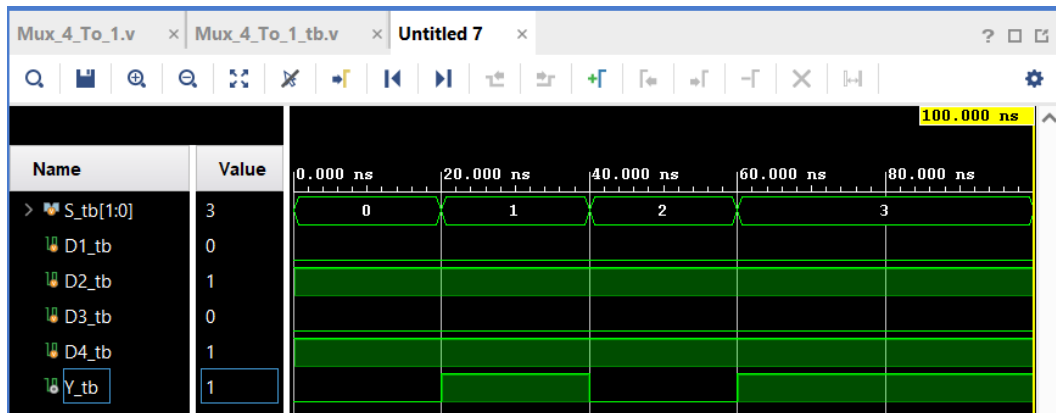


Figure 15: 4:1 Multiplexer simulation results.